

Aurora 900 SDK 开发者指南

V1.7 - 03/04/2025

目录

1 概述	6
2 安装说明	7
2.1 硬件安装	7
2.2 Aurora SDK 安装	7
2.2.1 编译环境配置	7
2.2.2 SDK 依赖环境配置	7
2.2.3 其它文档	8
3 Aurora 900 SDK 调用流程说明	9
3.1 接口调用流程	9
3.2 并发使用说明	9
4 SDK 使用实例	10
4.1 打开相机	10
4.2 创建/开启流	11
4.3 获取图像	11
4.4 关闭视频流	11
4.5 关闭相机	12
5 SDK 接口说明	13
5.1 头文件说明	13
5.2 设备模块说明	13
5.2.1 获取设备管理器对象	13
5.2.2 获取设备列表	13
5.2.3 注册设备插拔回调	14
5.2.4 创建设备对象	14
5.2.5 通过 usb 口创建设备	14
5.2.6 获取 SDK 版本号	15
5.2.7 获取设备对象类型	15

5.2.8 设置日志输出开关	15
5.2.9 打开设备	16
5.2.10 获取设置支持的图像帧格式	16
5.2.11 获取设置支持的流类型	16
5.2.12 设置 IR 帧率	17
5.2.13 设置设备图像帧格式	17
5.2.14 关闭设备	17
5.2.15 重启设备	18
5.2.16 升级设备	18
5.2.17 创建流对象	18
5.2.18 销毁流对象	19
5.2.19 Rgbdir 对齐开关(仅支持 AURORA930)	19
5.2.20 设置去除噪声设置阈值大小	19
5.2.21 获取设备产品序列号	20
5.2.22 获取摄像头的内外参	20
5.2.23 获取设备信息	20
5.2.24 重新启动设备	21
5.2.25 关停设备	21
5.2.26 获取当前设备状态	21
5.2.27 获取设备版本信息	21
5.2.28 深度/红外图像镜像 (aurora900 设备可用)	22
5.2.29 软件复位设备	22
5.2.30 设置自动曝光模式	22
5.2.31 获取自动曝光模式	23
5.2.32 开关自动曝光功能	23
5.2.33 设置曝光时间	24
5.2.34 获取曝光时间	24
5.2.35 设置传感器增益	24
5.2.36 获取传感器增益	25
5.2.37 设置室内外模式 (仅支持 AURORA930)	25
5.2.38 设置深度过滤范围 (仅支持 AURORA930)	25
5.2.39 深度矫正开关 (仅支持 AURORA930)	26
5.2.40 设置人脸曝光区域	26
5.2.41 获取设备支持的信息	26
5.2.42 获取温度信息	27

5.2.43 回滚固件版本	27
5.2.44 设置深度边缘裁边大小	27
5.2.45 设置深度距离	28
5.2.46 设置心跳	28
5.2.47 停止心跳	29
5.2.48 设置为扫码模式	29
5.2.49 设置图像分辨率	29
5.2.50 获取激光器电流	30
5.2.51 设置激光器电流	30
5.2.52 设置 Laser/Led 灯开关	30
5.3 流模块说明	31
5.3.1 启动流	31
5.3.2 停止流	31
5.3.3 获取单/多帧图像	31
5.3.4 注册回调接口	32
6 结构体及枚举类型说明	32
6.1 基础类型	32
6.1.1 流类型	32
6.1.2 帧类型	33
6.1.3 单帧	33
6.1.4 多帧	34
6.1.5 设备信息	34
6.2 设备类型	35
6.2.1 摄像机组件类型	35
6.2.2 摄像机 ID	35
6.2.3 设备信息	35
6.2.4 版本信息	36
6.2.5 帧数据	36
6.2.6 摄像机分辨率 (RGB 相机)	36
6.3 深度计算类型	37
6.3.1 摄像机内参	37
6.3.2 摄像机外参	37
6.3.3 空间点坐标	37

6.3.4 点云坐标及相应的 rgb 和 ir 数据	38
免责声明.....	40
技术支持.....	40

1 概述

Aurora SDK 是基于 Deptrum Aurora RGBD 模组开发的软件开发工具包，为应用开发者提供一系列友好的 API 和简单的应用示例程序。用户可以基于该开发包，获取高精度的深度图像、灰度图像和彩色图像信息，方便用户开发人脸识别、三维重建、人工智能感知等应用。本文档介绍了 Aurora SDK(C++) 的使用方法。包括使用流程、接口详解和例子程序使用。

2 安装说明

2.1 硬件安装

Aurora 3D 相机通过连接线与 PC 端的 USB 接口连接。

Windows 系统下，模组连接成功后，Windows 设备管理器出现 USB Lite -U20L，如图 1，



图 1 设备名称

Linux 系统下，在终端中运行“lsusb”，出现 vid 为 1900 的设备，说明模组连接成功。

2.2 Aurora SDK 安装

2.2.1 编译环境配置

- 安装软件包 build-essential

1. `sudo apt-get install build-essential`

`build-essential` 作用是提供编译程序必须软件包的列表信息。

- 安装跨平台编译工具 CMake

1. `sudo apt-get install cmake`

注意：本 sdk 需安装 **cmake 3.13.x** 及以上版本。

2.2.2 SDK 依赖环境配置

- 检查或安装 libusb

1. `sudo apt-get install libusb-dev`
2. `sudo apt-get install libusb-1.0-0-dev`

- 检查或安装 opengl 和 glfw 库

1. `sudo apt-get install libgl1-mesa-dev`
2. `sudo apt-get install build-dep glfw`
3. `sudo apt-get install libglu1-mesa-dev`
4. `sudo apt-get install libxi-dev`

2.2.3 其它文档

目录结构：参见 SDK 包 InstallDriver.txt

samples 编译指南：参见 SDK 包 samples/README.md

3 Aurora 900 SDK 调用流程说明

3.1 接口调用流程

从 Stream 获取 Frame 可以通过主动获取和被动获取两种方式获取。

- 主动获取通过 `GetFrames` 函数，获取图像；
- 被动获取通过 `RegisterFrameCb` 函数设置回调接口，获取图像。

SDK 流程如图 2 所示

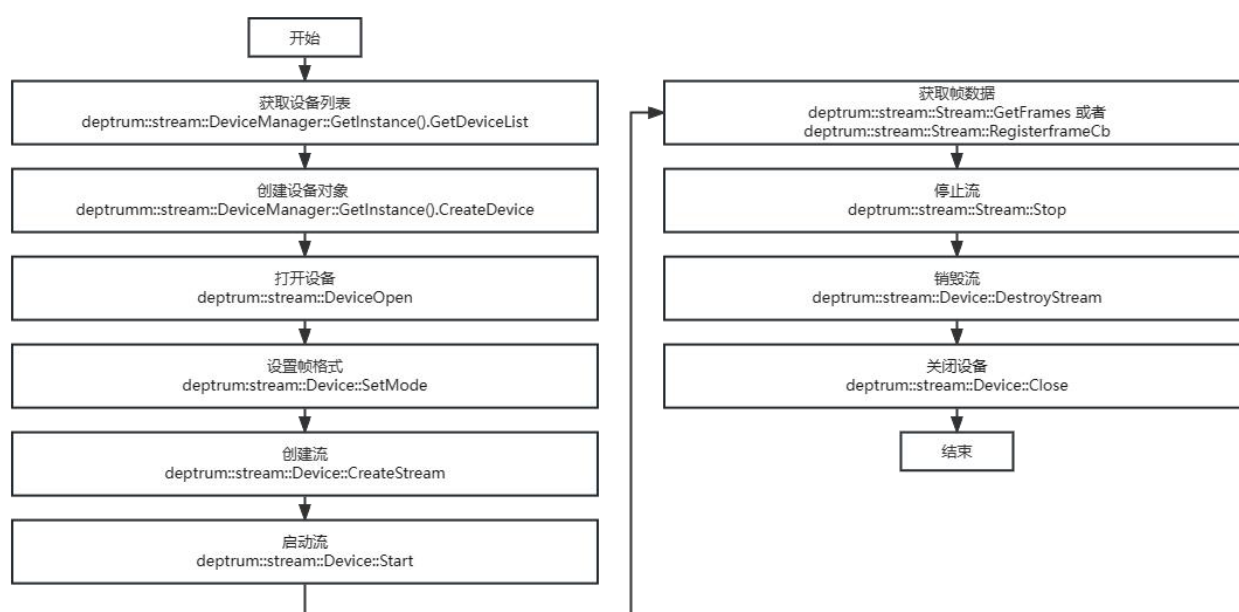


图 2 接口调用流程

3.2 并发使用说明

本 SDK 的所有函数接口都是线程安全的，支持多设备、单设备多流。

4 SDK 使用实例

4.1 打开相机

deptrum::stream 提供了一个 Aurora 3D 相机设备连接系统的接口，它提供访问设备、设备相关事件、版本和错误信息等功能，从 **device** 可以创建并访问 **streams**。

```
/****** get device list *****/
std::vector<DeviceInformation> device_list;
int ret = GetInstance()->GetDeviceList(device_list);
CHECK_SDK_RETURN_VALUE(ret);
```

```
/****** check device count *****/
CHECK_DEVICE_COUNT(device_list.size());
```

```
/****** create instance & open device *****/
std::shared_ptr<Device> device = GetInstance()-
>CreateDevice(device_list[0]);
CHECK_DEVICE_VALID(device);
```

```
ret = device->Open();
CHECK_SDK_RETURN_VALUE(ret);
```

```
FrameMode ir_mode, rgb_mode, depth_mode;
std::vector<std::tuple<FrameMode, FrameMode, FrameMode>>
device_resolution_vec;
device->GetDeviceSupportedFrameMode(device_resolution_vec);
```

```
ir_mode = std::get<0>(device_resolution_vec[0]);
rgb_mode = std::get<1>(device_resolution_vec[0]);
depth_mode = std::get<2>(device_resolution_vec[0]);
ret = device->SetMode(ir_mode, rgb_mode, depth_mode);
CHECK_SDK_RETURN_VALUE(ret);
```

4.2 创建/开启流

deptrum::Stream 从一个设备（Device）里提取一个视频流，从 stream 可以获取相应视频帧。
本 SDK 支持的 stream 类型请查看 include 目录下 deptrum/stream_types.h 中的 StreamType。

```
Stream* stream = nullptr;
int ret = device->CreateStream(stream, stream_type);
CHECK_SDK_RETURN_VALUE(ret);
ret = stream->Start();
```

4.3 获取图像

deptrum::Frame 从相关的 stream 里提取一个 frame。本 SDK 支持的 frame 类型与 stream 类型绑定，其数据格式如下表：

表 1 AuroraSDK 各 Stream 的 Frame 数据类型

Stream/Frame 类型	数据结构
红外图: kIr	uint8
深度图: kDepth	uint16 (mm)
红外-深度同出: kDepthIr	uint8 + uint16 (mm)
彩色图: kRgb	3 通道 uint8
RGBD 图: kRgbD	RGB 图+深度图
RGBD 图-红外同出: kRgbDlr	RGB 图+深度图+IR 图
点云图: kPointDense	PointXYZRgbIrr<float>

从 Stream 获取 Frame 可以通过两种方式：轮询和事件。

例如调用 GetFrames 函数主动获取帧数据：

```
StreamFrames frames;
ret = stream->GetFrames(frames, 2000);
```

4.4 关闭视频流

停止视频流，释放内存。

```
stream->Stop();  
device->DestroyStream(stream);
```

4.5 关闭相机

```
device->Close();
```

5 SDK 接口说明

5.1 头文件说明

头文件	内容说明
<code>deptrum/device.h</code>	设备相关公共功能接口
<code>deptrum/aurora900_series.h</code>	Aurora 900 系列设备特有的功能接口
<code>deptrum/stream.h</code>	流相关功能接口
<code>deptrum/stream_types.h</code>	流和设备版本相关结构体
<code>deptrum/stream_visualizer.h</code>	处理 depth 数据接口
<code>deptrum/common_types.h</code>	内外参相关结构体
<code>functional/base.h</code>	错误码、判断返回值以及打印标定参数
<code>functional/frame_rate_helper.h</code>	打印 fps

5.2 设备模块说明

5.2.1 获取设备管理器对象

```
static std::shared_ptr<DeviceManager> GetInstance();
```

通过该接口获取设备管理器的一个实例，设备管理器的对象是一个智能指针类型（`std::shared_ptr<DeviceManager>`），它封装了对设备管理器的访问。通过该对象，开发者可以访问设备的各项功能。

参数：

无

返回：

设备管理器的对象。

5.2.2 获取设备列表

```
virtual int GetDeviceList(std::vector<DeviceInformation>& device_list) = 0;
```

该方法用于获取当前系统中已连接的所有设备列表。开发者通过传递一个 `DeviceInformation` 类型的向量（`std::vector<DeviceInformation>& device_list`）给该方法，该方法会填充该向量，使其包含系统中每个已连接设备的信息。`DeviceInformation` 结构体通常包含设备的唯一标识符、设备名称、设备类型等关键信息。返

回值为整型（int），用于表示操作的成功与否，返回 0 表示成功，非 0 值表示失败。

参数：

[out]	device_list	设备列表的数组
-------	--------------------	---------

返回：

0：操作成功
非 0：操作失败

5.2.3 注册设备插拔回调

```
virtual void RegisterDeviceConnectedCallback(
    std::function<void(int flag, const DeviceInformation&
device_information)> handle, bool enable_hotplug
) = 0;
```

注册设备插拔回调，该回调机制允许应用程序在设备连接或断开时接收到相应的通知。通过注册一个回调函数，开发者可以指定当设备插拔事件发生时，应执行的操作。其中，flag 参数用于标识插拔事件的类型，例如，连接或断开。device_information 参数则包含了插拔设备的详细信息，如设备名称、类型等。

enable_hotplug 参数用于启用或禁用热插拔功能，当设置为 true 时，系统将允许设备的热插拔操作，并在设备插拔时触发回调函数。

参数：

[out]	flag	1: 设备插入, 2: 设备拔出
[out]	device_information	设备对象信息
[in]	enable_hotplug	是否开启热插拔

返回：

5.2.4 创建设备对象

```
virtual std::shared_ptr<Device> CreateDevice(const DeviceInformation&
device_information) = 0;
```

该函数用来创建设备对象，在调用该函数时，需要传入摄像头设备的详细信息，这些信息通过 DeviceInformation 结构体来提供。成功创建设备对象后，开发者可以进一步利用该对象进行摄像头的配置、启动预览、捕获图像或视频等操作。

参数：

[in]	device_information	设备对象信息
------	---------------------------	--------

返回：

返回设备对象的智能指针

5.2.5 通过 usb 口创建设备

```
virtual std::shared_ptr<Device> CreateDeviceByUsbPort(std::string usb_port) =
0;
```

通过 USB 端口直接创建设备对象，此方法允许开发者在已知设备连接的 USB 端口时，快速实例化设备对象，进行后续的操作和控制。通过指定 USB 端口号，系统内部会查找并验证该端口上的设备，一旦确认设备存在

且符合创建条件，便会返回相应的设备对象智能指针。

参数：

[in]	usb_port	usb 端口号，DeviceInformation 中 port_path 描述的字段。
------	-----------------	--

返回：

返回设备对象的智能指针。

5.2.6 获取 SDK 版本号

```
virtual std::string GetSdkVersion() = 0;
```

获取当前使用的 SDK 版本信息

返回：

SDK 版本号

5.2.7 获取设备对象类型

```
virtual std::string GetDeviceName() = 0;
```

该方法用于获取当前设备对象的名称，便于开发者识别和管理不同的设备对象。

参数：

无

返回：

设备类型字符串。

5.2.8 设置日志输出开关

```
virtual void EnableLogging(const std::string& path, bool enable)=0;
```

该功能允许开发者控制是否将设备的日志信息输出到指定的文件路径。当 enable 参数为 true 时，日志信息将被写入到 path 参数指定的文件中；当 enable 参数为 false 时，将停止向该文件写入日志信息。这对于调试和监控设备运行状态非常有用，开发者可以根据需要开启或关闭日志记录，以便更好地分析和解决问题。

参数：

[in]	path	文件输出路径
[in]	enable	文件输出控制位，true：输出到文件，false：关闭输出

返回：

无

5.2.9 打开设备

```
virtual int Open()=0;
```

打开设备，开发者在调用其他依赖于设备连接状态的函数之前，应确保设备已经成功打开。返回值为整型，通常用于表示操作的成功与否，0 表示成功，非 0 值表示失败。

参数：无

返回：

0：操作成功
非 0：操作失败

5.2.10 获取设置支持的图像帧格式

```
virtual int GetDeviceSupportedFrameMode(
    std::vector<std::tuple<FrameMode, FrameMode, FrameMode>>&
    device_resolution_vec) = 0;
```

该函数用于获取设备支持的图像帧格式，以便开发者在编程时选择合适的帧格式。返回的 device_resolution_vec 参数包含了支持的帧格式。开发者可以通过遍历这个向量，了解设备支持的所有帧格式，并根据实际需求选择合适的帧格式进行后续的开发工作。对于 aurora900 设备，当前无需手动设置帧格式，设备会根据实际情况自动选择合适的帧格式进行工作。

参数：

[out] device_resolution_vec 存储支持的帧格式

返回：

0：操作成功
非 0：操作失败

5.2.11 获取设置支持的流类型

```
virtual int GetDeviceSupportedStreamType(std::vector<StreamType>&
    device_streamtype_vec)
    = 0;
```

获取设备支持的流类型信息。device_streamtype_vec 参数是一个向量，用于存储设备支持的所有流类型。开发者可以遍历这个向量，了解设备支持的所有流类型，以便在后续的开发工作中根据实际需求选择合适的流类型。

参数：

[out] device_streamtype_vec 支持的开流类型

返回：

0：操作成功
非 0：操作失败

5.2.12 设置 IR 帧率

```
virtual int SetIrFps(int ir_fps) = 0;
```

设置设备的 IR 帧率，注意 RGB 帧率也会同步生效。通过 ir_fps 参数，开发者可以指定 IR 帧率的具体值。

参数：

[in]	ir_fps	ir 帧率，aurora930 只支持 5 和 10 和 12 和 15 fps
------	---------------	--

返回：

0：操作成功
非 0：操作失败

5.2.13 设置设备图像帧格式

```
virtual int SetMode(const FrameMode& ir_mode = kInvalid,
                   const FrameMode& rgb_mode = kInvalid,
                   const FrameMode& depth_mode = kInvalid,
                   FrameDecodeMode ir_decode_mode = kSoftwareDecode,
                   FrameDecodeMode rgb_decode_mode = kSoftwareDecode) = 0;
```

由于 aurora900 设备当前无需设置帧格式，可直接调用 SetMode()方式，使用默认值。

参数：

[in]	ir_mode	IR 图像帧格式
[in]	rgb_mode	RGB 图像帧格式
[in]	depth_mode	Depth 图像帧格式
[in]	ir_decode_mode	IR 图像帧解码模式
[in]	rgb_decode_mode	RGB 图像帧解码模式

返回：

0：操作成功
非 0：操作失败

5.2.14 关闭设备

```
virtual int Close() = 0;
```

该方法用于关闭 aurora900 设备，释放相关资源。调用此方法后，设备将停止工作，无法再接收或处理图像数据。在调用此方法之前，请确保已完成所有必要的操作。

参数：无

返回：

0：操作成功
非 0：操作失败

5.2.15 重启设备

```
virtual int Reboot() = 0;
```

该方法用于重启设备，使其恢复到初始状态。重启后，设备将重新加载配置并启动，之前的状态和数据将被清除。

返回：

0：操作成功
非 0：操作失败

5.2.16 升级设备

```
virtual int Upgrade(const std::string& update_file_path,  
                   ProgressHandler progress_handler = nullptr) = 0;
```

该方法允许用户通过指定的固件升级文件对设备进行固件升级。升级过程中，可以通过 progress_handler 回调函数获取升级进度，以使用户了解升级状态。如果 progress_handler 为 nullptr，则不会报告进度。

参数：

[in]	update_file_path	升级文件的路径
[in]	progress_handler	回调接口，返回升级进度。

返回：

0：操作成功
非 0：操作失败

5.2.17 创建流对象

```
virtual int CreateStream(Stream*& stream, const std::vector<StreamType>& types)  
= 0;
```

创建流对象功能允许用户根据需求指定不同类型的流。Stream*参数是一个指向流对象的指针，通过此指针可以访问和操作创建的流。const std::vector<StreamType>& types 参数是一个包含所需流类型的向量，用户可以根据需要向此向量中添加一个或多个流类型。设备驱动程序会根据提供的类型尝试创建一个或多个流对象，并将它们的指针赋值给 stream 参数。如果创建成功，函数返回 0 表示操作成功；如果创建失败，则返回非 0 值表示操作失败。

参数：

[in/out]	stream	流对象指针引用
[in]	types	流的类型, 可多个

返回：

0：操作成功
非 0：操作失败

注意：

流对象指针引用

5.2.18 销毁流对象

```
virtual int DestroyStream(Stream*& stream) = 0;
```

销毁已经创建的流对象以释放资源，可销毁指定类型的流对象。销毁流对象时，会释放与该流对象相关的资源。

参数：

[in]	stream	流对象指针引用
------	---------------	---------

返回：

0：操作成功
非 0：操作失败

5.2.19 Rgbdir 对齐开关(仅支持 AURORA930)

```
virtual int SwitchAlignedMode(bool mode = false) = 0;
```

Rgbdir 对齐开关用于调整 AURORA930 设备的 RGBDIR 数据对齐方式。当调用 SwitchAlignedMode 函数并传入 true 参数时，设备将启用对齐模式；传入 false 参数时，设备将关闭对齐模式。SetMode 之后调用此功能，可以确保在对齐模式切换前，设备的当前状态已经按照预期进行了配置。

参数：

[in]	mode	默认 false，true 是打开对齐功能
------	-------------	-----------------------

返回：

0：操作成功
非 0：操作失败

5.2.20 设置去除噪声设置阈值大小

```
virtual int SetRemoveFilterSize(uint32_t size = 110) = 0;
```

用于设置在图像处理过程中，哪些噪声点将被去除。设置一个合适的阈值大小，可以有效减少图像中的噪声干扰，同时保留图像的重要细节。如果阈值设置过低，可能会导致一些重要的图像细节被误判为噪声而被去除；反之，如果阈值设置过高，则可能无法有效去除图像中的噪声。可根据实际应用场景和需求，合理设置去除噪声的阈值，SetMode 之后调用即可。

参数：

[in]	Size	默认 110
------	-------------	--------

返回：

0：操作成功
非 0：操作失败

5.2.21 获取设备产品序列号

```
virtual int GetSerialNumber(std::string *sn) = 0;
```

该接口用于获取当前设备的唯一产品序列号，以便于设备的识别和管理。调用该接口后，设备的产品序列号将被存储在传入的字符串指针参数中。

参数：

[out]	sn	产品序列号
-------	-----------	-------

返回：

0：操作成功
非 0：操作失败

5.2.22 获取摄像头的内外参

```
virtual int GetCameraParameters(Intrinsic& ir_intri, Intrinsic& rgb_intri, Extrinsic& ext) = 0;
```

该接口用于获取当前设备摄像头的内参和外参。内参包括摄像头的焦距、主点坐标等内部参数，而外参则描述了摄像头相对于某个参考坐标系的位置和姿态。通过获取这些参数，可以对摄像头采集的图像进行精确的校正和定位。调用该接口后，摄像头的内参将被存储在传入的 ir_intri 和 rgb_intri 参数中，外参将被存储在传入的 ext 参数中。

参数：

[out]	ir_intri	Ir 摄像头内部参数
[out]	rgb_intri	Rgb 摄像头内部参数
[out]	ext	摄像头外部参数

返回：

0：操作成功
非 0：操作失败

5.2.23 获取设备信息

```
virtual void GetDeviceInfo(DeviceInformation& device_information) = 0;
```

该接口用于获取设备的详细信息，包括设备的地址、制造商、名字、端口号等。调用该接口后，设备信息将被填充到传入的 device_information 参数中。

参数：

[out]	device_information	设备信息
-------	---------------------------	------

返回：

无

5.2.24 重新启动设备

```
virtual int RestartDevice(const DeviceInformation device_information)
    = 0;
```

该接口主要用于在设备出现异常情况或需要重新启动时，通过传入的设备信息参数重新启动设备和恢复数据流。通常情况下，不建议主动调用。

参数：

[in]	device_information	设备信息
------	--------------------	------

返回：

0：操作成功

非 0：操作失败

5.2.25 关停设备

```
virtual int StopDevice(const DeviceInformation& device_information)
    = 0;
```

调用此接口后，设备将停止出流并且关闭设备，不建议主动调用。

参数：

[in]	device_information	设备信息
------	--------------------	------

返回：

0：操作成功

非 0：操作失败

5.2.26 获取当前设备状态

```
virtual bool GetDeviceStatus() = 0;
```

该接口用于获取当前设备的状态信息。返回值为布尔类型，若返回 true，表示成功获取到设备状态；若返回 false，则可能表示获取设备状态失败或设备当前无法提供状态信息。

参数：无

返回：

True: 为设备开启

False: 设备关闭

5.2.27 获取设备版本信息

```
virtual int GetDeviceInfo(DeviceDescription& device_info)
    = 0;
```

获取设备信息，包括：

device_name 表示设备名称。

serial_num 为设备的序列号，用于唯一标识该设备。

stream_sdk_version 表示 SDK 的版本号。

rgb_firmware_version 和 ir_firmware_version 分别表示 RGB 和红外固件的版本号。

pid (Product ID) 和 vid (Vendor ID) 是设备的硬件标识符。

参数：

[out]	device_info	产品版本信息，参考 DeviceDescription
-------	-------------	-----------------------------

返回：

0：操作成功

非 0：操作失败

5.2.28 深度/红外图像镜像（aurora900 设备可用）

```
virtual int EnableDepthIrMirroring(bool enable) = 0;
```

镜像深度图像和红外图像，通过调用 EnableDepthIrMirroring 方法，开发者可以启用或禁用此功能。当 enable 参数为 true 时，深度图像和红外图像将被镜像处理，即图像内容在水平方向上左右翻转；当 enable 参数为 false 时，则关闭镜像功能，图像保持原始状态。

参数：

[in]	enable	是否开启深度镜像 true 开启 false 关闭
------	--------	---------------------------

返回：

0：操作成功

非 0：操作失败

5.2.29 软件复位设备

```
virtual int SoftResetDevice() = 0;
```

在某些情况下，设备可能需要执行软件复位操作以恢复其初始状态或解决运行异常。SoftResetDevice 方法提供了一种便捷的方式来实现这一目标。返回 0 表示软件复位操作成功完成；返回非 0 值则表示操作失败。

参数：无

返回：

0：操作成功

非 0：操作失败

5.2.30 设置自动曝光模式

```
virtual int SetAEMode(const CameraComponent& component, uint8_t mode) = 0;
```

设置自动曝光模式，该方法允许用户指定相机组件和所需的曝光模式。参数 component 指定了要设置曝光

模式的相机组件，而 **mode** 参数则定义了具体的曝光模式。返回 0 表示自动曝光模式设置成功；返回非 0 值则表示操作失败。

参数：

[in]	component	传感器种类
[in]	mode	自动曝光模式

返回：

0：操作成功
非 0：操作失败

5.2.31 获取自动曝光模式

```
virtual int GetAEMode(const CameraComponent& component, uint8_t& mode) = 0;
```

该方法用于检索指定相机组件的当前自动曝光模式。参数 **component** 指定了要查询曝光模式的相机组件，**mode** 参数通过引用传递，用于存储检索到的曝光模式值。返回 0 表示成功检索到自动曝光模式；返回非 0 值则表示操作失败。

注意：speckle 除外。

参数：

[in]	component	传感器种类
[out]	mode	自动曝光模式

返回：

0：操作成功
非 0：操作失败

5.2.32 开关自动曝光功能

```
virtual int SwitchAutoExposure(bool enable) = 0;
```

该方法用于控制相机的自动曝光功能。参数 **enable** 为布尔类型，用于指定是否启用自动曝光功能。当 **enable** 为 true 时，启用自动曝光功能；当 **enable** 为 false 时，禁用自动曝光功能。

参数：

[in]	enable	True 开启 IR 自动曝光，False 关闭 IR 自动曝光，手动设置曝光参数前需要将其关闭。
--------	---------------	---

返回：

0：操作成功
非 0：操作失败

5.2.33 设置曝光时间

```
virtual int SetExposure(const CameraComponent& component, int exposure_in_us)=0;
```

该接口允许用户为指定的相机组件设置曝光时间。参数 `component` 代表相机组件的标识，`exposure_in_us` 代表曝光时间，单位为微秒。曝光时间的设置范围在 1 到 31 之间，每个数值代表 0.1 毫秒的曝光时间。例如，如果需要将曝光时间设置为 2.5 毫秒，则应传入 25 作为 `exposure_in_us` 的值。

请注意，在调用此接口之前，用户应确保已经正确禁用了自动曝光功能，否则设置的曝光时间可能会被自动曝光算法覆盖。此外，曝光时间的设置可能受到相机模组硬件的限制，如果设置的曝光时间超出了硬件支持的范围，可能会导致操作失败或相机工作异常。

返回值为 0 表示操作成功，非 0 值表示操作失败。在调用此接口后，用户可以通过相应的接口获取当前的曝光时间设置，以确认是否设置成功。

说明：

参数：

[in]	component	传感器种类
[in]	exposure_in_us	曝光时间

返回：

0：操作成功
非 0：操作失败

5.2.34 获取曝光时间

```
virtual int GetExposure(const CameraComponent& component, int& exposure_in_us)=0;
```

获取模组中指定传感器的曝光时间，speckle 图暂不支持。

参数：

[in]	component	传感器种类
[out]	exposure_in_us	曝光时间

返回：

0：操作成功
非 0：操作失败

5.2.35 设置传感器增益

```
virtual int SetGain(const CameraComponent& component, uint32_t gain) = 0
```

该函数用于调整相机模组中指定传感器的增益值，以改善图像质量。增益值越高，图像亮度越高，但噪点也可能相应增加。

参数：

[in]	component	传感器种类
[in]	gain	增益的值

返回：

0：操作成功

非 0：操作失败

5.2.36 获取传感器增益

```
virtual int GetGain(const CameraComponent& component, uint32_t& gain) = 0
```

该函数用于读取相机模组中指定传感器的当前增益值。通过调用该函数，用户可以获取到当前传感器设置的增益值，以便进一步分析或调整图像质量。

参数：

[in]	component	传感器种类
[out]	gain	增益的值

返回：

0：操作成功
非 0：操作失败

5.2.37 设置室内外模式（仅支持 AURORA930）

```
virtual int SetLaserDriver(float laser_driver) = 0
```

该函数允许用户为相机模组设置室内或室外模式。在不同的光照条件下，室内外模式可以帮助优化相机的图像捕捉性能。例如，在室外强光环境下，相机可能会调整其曝光和增益设置，以减少过度曝光并保持图像细节；而在室内低光环境下，相机可能会增加增益以提高图像的亮度。通过调用该函数，用户可以根据当前环境选择合适的模式，从而提升图像质量和相机的适应性。

参数：

[in]	laser_driver	室内外模式，1-auto 2-indoor 3-outdoor
------	---------------------	---------------------------------

返回：

0：操作成功
非 0：操作失败

5.2.38 设置深度过滤范围（仅支持 AURORA930）

```
virtual int FilterOutOfRangeDepthMap(int min = 150, int max = 4000) = 0
```

该函数允许用户过滤掉设定深度范围之外的深度数据，只保留在指定深度范围内的数据。通过调整最小和最大深度值，用户可以精确地控制深度数据的范围，从而优化后续的 3D 重建、物体识别等应用。

参数：

[in]	min	深度范围最小值
[in]	max	深度范围最大值

返回：

0：操作成功
非 0：操作失败

5.2.39 深度矫正开关（仅支持 AURORA930）

```
virtual int DepthCorrection(bool depth_correction) = 0
```

该函数用于开关设备的深度数据校准功能，通过启用深度矫正功能，设备能够自动调整深度数据中的误差，使得生成的 3D 图像和物体识别结果更为精确。

参数：

[in]	depth_correction	True: 打开深度校准 False: 关闭深度校准
------	-------------------------	----------------------------

返回：

0: 操作成功
非 0: 操作失败

5.2.40 设置人脸曝光区域

```
virtual int SetFaceArea(DeptrumBBBox face_area) = 0;
```

该函数允许用户自定义人脸曝光区域，通过调整坐标参数，可以优化人脸在图像中的亮度和对比度，从而提高人脸识别的准确性和稳定性。

参数：

[in]	face_area	人脸曝光区域
------	------------------	--------

返回：

0: 操作成功
非 0: 操作失败

5.2.41 获取设备支持的信息

```
virtual int GetSupportInfo(SupportedInfo& support_info) = 0;
```

获取设备可以支持相关功能信息，如下：

scan_face_mode: 表示设备是否支持人脸扫描模式，1 表示支持，0 表示不支持。

scan_code_mode: 表示设备是否支持条码扫描模式，1 表示支持，0 表示不支持。

running_7x24_hours: 表示设备是否支持 7x24 小时不间断运行，1 表示支持，0 表示不支持。

depth_range: 表示设备的深度范围信息，包含最小深度、最大深度等具体参数。

is_support_se: 表示设备是否支持安全加密功能，1 表示支持，0 表示不支持。

is_support_synced_2_img: 表示设备是否支持两张图像同步的功能，1 表示支持，0 表示不支持。

参数：

[out]	support_info	支持的信息内容
-------	---------------------	---------

返回：

0: 操作成功
非 0: 操作失败

5.2.42 获取温度信息

```
virtual int GetCameraTemperature (TemperatureType temp_type, int16_t* temperature) = 0;
```

获取模组内部各模块温度信息，目前支持：主板温度、激光器温度、CPU 温度

参数：

		获取温度模块：
[in]	temp_type	TemperatureCamera = 1, TemperatureVcsel, TemperatureCpu,
[out]	temperature	模块温度

返回：

0：操作成功
非 0：操作失败

5.2.43 回滚固件版本

```
virtual int RollBack() = 0;
```

此功能允许用户将设备的固件版本回滚到上一个版本。在某些情况下，如果当前固件版本存在不稳定或兼容性问题，回滚固件版本可能是一个有效的解决方案。通过调用 RollBack 函数，用户可以触发回滚过程。请注意，在执行回滚操作前，建议备份当前固件，以防止数据丢失或设备无法正常工作。回滚操作完成后，设备将重启并加载上一个固件版本。

参数：

返回：

0：操作成功
非 0：操作失败

5.2.44 设置深度边缘裁边大小

```
virtual int SetFilterOutBorderDepth(int border_depth) = 0;
```

此功能允许用户调整设备在进行深度测量时边缘裁边的大小。边缘裁边是指在深度测量过程中，设备会忽略掉边缘区域的数据，以减少因边缘效应引起的测量误差。通过设置 border_depth 参数，用户可以控制被忽略的边缘区域的大小。

参数：

[in]	border_depth	深度裁边大小，范围为[0-15]
--------	--------------	------------------

返回：

0：操作成功
非 0：操作失败

5.2.45 设置深度距离

```
virtual int SetDepthRange(int min_range, int max_range) = 0;
```

此功能允许用户定义设备在进行深度测量时的最小和最大测量范围。通过设置 min_range 和 max_range 参数，用户可以限制设备只对特定深度范围内的数据进行测量。

参数：

[in]	min_depth	最小深度范围
[in]	max_depth	最大深度范围

返回：

0：操作成功
非 0：操作失败

5.2.46 设置心跳

```
virtual int SetHeartbeat(const HeartbeatParam& heartbeat_params,  
                          HeartbeatResult heartbeat_callback = nullptr) = 0;
```

该功能用于配置设备的心跳机制，以便定期发送心跳信号以维持与设备的连接或监控设备的状态。HeartbeatParam 结构体包含了心跳信号参数配置，如心跳间隔时间、心跳信号的数据内容等。heartbeat_callback 是一个回调函数，当设备接收到心跳信号时，该函数将被调用，并可以处理心跳信号或执行相关操作。如果不需要处理心跳信号，可以将 heartbeat_callback 参数设置为 nullptr。通过设置心跳机制，用户可以确保设备的连接稳定性，及时发现设备异常情况。

说明：

针对不支持心跳的设备，若开启，则会收到心跳失败的回调。

参数：

[in]	heartbeat_params	心跳参数，包括超时时间，允许失败次数和是否开启
[in]	heartbeat_callback	心跳回调

返回：

0：操作成功
非 0：操作失败

5.2.47 停止心跳

```
virtual int StopHeartbeat() = 0;
```

当设备不再需要心跳机制进行连接稳定性检测时，可以调用 StopHeartbeat 函数来停止心跳。该函数不接受任何参数，执行后将关闭当前的心跳机制，停止发送和接收心跳信号。停止心跳后，设备将不再触发心跳回调，用户也无法再通过心跳机制检测设备的连接状态。

返回：

0：操作成功
非 0：操作失败

5.2.48 设置为扫码模式

```
virtual int EnableScanCodeMode(bool enable) = 0;
```

当需要将设备设置为扫码模式时，可以调用 EnableScanCodeMode 函数。该函数接受一个布尔类型的参数 enable，用于指定是否启用扫码模式。当 enable 为 true 时，设备将进入扫码模式。

参数：

[in]	enable	True：开启扫码模式，False：关闭扫码模式
--------	---------------	--------------------------

返回：

0：操作成功
非 0：操作失败

5.2.49 设置图像分辨率

```
virtual int SetStreamImageResolution(StreamImageResolution  
                                     image_resolution) = 0;
```

设置图像分辨率

可以通过调用该函数来设置图像分辨率。StreamImageResolution 是一个枚举类型，定义了设备支持的不同图像分辨率选项。

注意：

目前只支持 rgb 图像 分辨率调整，支持以下分辨率：768x432/1920x1080

参数：

[in]	image_resolution	图像分辨率
--------	-------------------------	-------

返回：

0：操作成功
非 0：操作失败

5.2.50 获取激光器电流

```
virtual int GetLaserCurrent(int& current) = 0;
```

该函数用于获取当前激光器的工作电流，单位为 mA。

参数：

[out]	current	电流值 (1800mA,1500mA,1600mA,1700mA,900mA, 2000mA,1000mA,1100mA,1200mA,1300mA,1400mA,500mA,600mA,700mA,800mA, 900mA, 1450mA),默认值是 1450mA
---------	----------------	--

返回：

0：操作成功
非 0：操作失败

5.2.51 设置激光器电流

```
virtual int SetLaserCurrent(int current) = 0;
```

该函数用于设置激光器的工作电流，单位为 mA。用户可以通过调用该函数，将激光器的工作电流调整为自己需要的值。在调用该函数时，用户需要传入一个 int 类型的参数 current，该参数指定了激光器的新工作电流值。函数执行成功后，激光器的工作电流将被设置为该值。如果函数执行失败，将返回一个非 0 值，表示操作失败。通过调整激光器的工作电流，用户可以控制激光器的输出功率，从而满足不同的应用需求。

参数：

[in]	current	电流值 (1800mA,1500mA,1600mA,1700mA,900mA, 2000mA,1000mA,1100mA,1200mA,1300mA,1400mA,500mA,600mA,700mA,800mA, 900mA, 1450mA),默认值是 1450mA
--------	----------------	--

返回：

0：操作成功
非 0：操作失败

注意：

设置电流后需重启之后才能生效

5.2.52 设置 Laser/Led 灯开关

```
virtual int SetLightMode(StreamLightMode light_mode) = 0;
```

设置 Laser/Led 灯开关

可以通过调用该函数来控制 Laser 和 Led 的开关模式。

参数：

[in] light_mode 灯模式; 1: all on; 2: all off; 3: laser on, led off; 4: laser off, led on

返回:

0: 操作成功
非 0: 操作失败

5.3 流模块说明

5.3.1 启动流

```
virtual int Start() = 0;
```

用于启动摄像头数据流。调用此函数后，系统将开始采集和传输摄像头的的数据。若函数调用成功，将返回 0，表示摄像头数据流已成功启动。用户可以在此基础上进行后续的数据处理或分析。请确保在调用此函数前，摄像头设备已正确连接并初始化。

参数: 无

返回:

0: 操作成功
非 0: 操作失败

5.3.2 停止流

```
virtual int Stop() = 0;
```

调用此函数后，系统将停止采集和传输摄像头的的数据。若函数调用成功，将返回 0，表示摄像头数据流已成功停止。停止数据流后，用户可以进行其他操作，如关闭摄像头设备或释放相关资源。

参数: 无

返回:

0: 操作成功
非 0: 操作失败

5.3.3 获取单/多帧图像

```
virtual int GetFrames(StreamFrames& frames, uint32_t timeout = -1) = 0;
```

此函数用于从摄像头设备获取单帧或多帧图像数据。用户可以指定一个超时时间 (timeout)，单位为毫秒。如果在指定的超时时间内未能获取到图像数据，函数将返回失败。若未设置超时时间 (即 timeout 为默认值 -1)，函数将一直阻塞，直到成功获取到图像数据。

StreamFrames 结构体用于存储获取到的图像帧数据，包括图像帧的数量、每帧图像的宽度、高度、像素格式等信息以及图像数据的实际存储位置。用户可以通过访问此结构体来获取并处理图像数据。

需要注意的是，在调用 GetFrames 函数期间，不应同时调用 RegisterFrameCb 函数进行图像数据的回调注册。两者是互斥的，同时调用可能会导致未定义的行为或错误。

参数:

[out]	frames	Frames 对象的引用
[in]	timeout	超时时间, 单位为毫秒

返回:

0: 操作成功
非 0: 操作失败

5.3.4 注册回调接口

```
virtual void RegisterFrameCb(std::function<int(StreamFrames&)> cb) = 0;
```

主动注册回调接口（与 **GetFrames** 接口互斥）

主动注册回调接口允许用户在图像帧数据到达时，通过注册的回调函数自动获取并处理这些数据，而无需主动调用 **GetFrames** 函数。

当注册回调接口后，每当有新的图像帧数据到达时，系统会自动调用用户提供的回调函数，并将包含图像帧数据的 **StreamFrames** 结构体作为参数传递给该函数。回调函数可以根据需要对图像数据进行处理，如显示、保存或进一步分析等。

需要注意的是，由于回调接口和 **GetFrames** 接口是互斥的，因此在注册回调接口后，不应再调用 **GetFrames** 函数来获取图像数据。否则，可能会导致未定义的行为或错误。

参数:

[in]	cb	回调函数指针
------	-----------	--------

返回:

0: 操作成功
非 0: 操作失败

6 结构体及枚举类型说明

6.1 基础类型

6.1.1 流类型

```
enum StreamType {
    kInvalidStreamType = 0, // Invalid stream
    kRgb = 1,               // Rgb stream
    kIr = 2,                // Ir stream
    kDepth = 3,             // Depth stream
    kRgbD = 4,              // RgbD stream, frames order in rgb depth
    kRgbIr = 5,              // RgbIr Stream, frames order in rgb ir
    kDepthIr = 6,           // DepthIr stream, frames order in depth ir
    kRgbDIr = 7,            // RgbD Ir stream, frames order in rgb depth ir
    kDepthIrLaser = 8,
    kSpeckleCloud = 9,
    kPointCloud = 10,       // Dense point cloud stream
}
```



```
kRgbDensePointCloudStream = 11, // RgbDense point cloud stream
kRgbDirIrDensePointCloudStream = 12, // RgbDir Ir Dense point cloud stream;
kRgbDirIrFlag = 13, // rgbdir+flag stream
}
```

6.1.2 帧类型

```
enum FrameType {
    kInvalidFrameType = 0, // Invalid frame
    kRgbFrame = 1, // RGB frame
    kIrFrame = 2, // IR frame
    kDepthFrame = 3, // Depth frame
    kLaserFrame = 4, // flag frame
    kPointCloudFrame = 5, // Dense point cloud frame
    kRgbDensePointCloudFrame = 6,
    kRgbDirIrPointCloudFrame = 7,
}
```

6.1.3 单帧

```
struct StreamFrame {
    int index; // Index of the frame
    int size; // Frame data size in bytes
    int cols; // Number of columns
    int rows; // Number of rows
    int bits_per_pixel; // Number of bits per pixel
    float temperature; // Driver temperature during this frame
    FrameType frame_type; // Type of the frame, refer to FrameType
    FrameAttr frame_attr; // Attributes of the frame, refer to FrameAttr
    ImageFormat image_format; // Format of the image
    uint64_t timestamp; // Timestamp of the frame
    std::shared_ptr<void> data; // Pointer to the frame data
};
```

参数:

frame_type	帧类型
index	帧索引
size	数据大小
cols	宽度
rows	高度
bits_per_pixel	每个像素位数
temperature	driver 温度
frame_attr	帧属性信息
data	数据指针
timestamp	时间戳

备注

depth 和 ir 流的每个像素的数据类型为 uint16_t，其中 depth 单位为 mm；depth-ir 流数据格式为：depth 图+ir 图，格式同上；point-cloud 流的每个点的坐标类型为 float，单位 mm；rgb 流的每个像素的颜色分量数据类型为 uint_8，顺序为 BGR。详细解析方式见 samples。

6.1.4 多帧

```
struct StreamFrames {
    int count; // Number of frames
    std::vector<std::shared_ptr<StreamFrame>> frame_ptr; // Pointer to the
frames array
    std::shared_ptr<ExtraFrameInfo> extra_info{}; // Extra frames
information
};
```

参数:

count	帧数量
frame_ptr	帧指针
extra_info	扩展信息

6.1.5 设备信息

```
struct DeviceDescription {
    std::string device_name;
    std::string serial_num;
    std::string stream_sdk_version;
    std::string rgb_firmware_version;
    std::string ir_firmware_version;
    uint16_t pid;
    uint16_t vid;
};
```

参数:

device_name	产品型号名
serial_num	产品序列号
stream_sdk_version	Stellar SDK 版本号
rgb_firmware_version	Rgb 摄像头固件版本
ir_firmware_version	Tof 摄像头固件版本
pid	Tof 摄像头产品 id
vid	Tof 摄像头厂家 id

6.2 设备类型

6.2.1 摄像机组件类型

```
enum CameraComponent {  
    kInvalidCameraComponent = 0,  
    kIrCamera    = 0,  
    kRgbCamera   = 1,  
    kLaser       = 2,  
    kLed         = 3,  
};
```

参数:

kIrCamera	IR 摄像机
kRgbCamera	RGB 摄像机
kLaser	激光
kLed	红外光

6.2.2 摄像机 ID

```
struct CameraComponentID {  
    uint8_t  device_addr = 0;  
    uint16_t vid = 0;  
    uint16_t pid = 0;  
    std::string serial_number;  
    std::string name;  
    std::string port_path;  
};
```

参数:

device_addr	设备地址
vid	厂家 ID
pid	产品 ID
serial_number	设备序列号
name	摄像机名称
port_path	USB 端口地址

6.2.3 设备信息

```
struct DeviceInformation {  
    std::string model;  
    CameraComponentID ir_camera;  
    CameraComponentID rgb_camera;  
};
```

参数:

model	产品型号
ir_camera	红外摄像机 ID
rgb_camera	RGB 摄像机 ID

6.2.4 版本信息

```
struct Version {
    uint8_t major;
    uint8_t minor;
    uint8_t revision;
    char tag[8];
};
```

参数:

major	主版本号
minor	副版本号
revision	修正版本号
tag[8]	标记

6.2.5 帧数据

```
struct FrameFormat {
    int rows;
    int cols;
    int frame_size;
    int bits_per_pixel;
};
```

参数:

rows	行数
cols	列数
frame_size	数据大小
bits_per_pixel	每个像素的大小

6.2.6 摄像机分辨率 (RGB 相机)

```
enum StreamImageResolution{
    kResolutionInvalid = 0,
    kResolution480x768,
    kResolution720x1280,
    kResolution1080x1920,
};
```

参数:

kResolutionInvalid	无效的分辨率
kResolution480x768	480x768

kResolution720x1280	720x1280
kResolution1080x1920	1080x1920

6.3 深度计算类型

6.3.1 摄像机内参

```
struct Intrinsic {
    int rows;
    int cols;
    float focal_length[2];
    float principal_point[2];
    float distortion_coeffs[5];
};
```

参数:

rows	行数
cols	列数
focal_length[2]	镜头焦距
principal_point[2]	主点
distortion_coeffs[5]	镜头畸变参数

6.3.2 摄像机外参

```
struct Extrinsic {
    float rotation_matrix[9];
    float translation_vector[3];
};
```

参数:

rotation_matrix[9]	旋转矩阵
translation_vector[3]	平移矩阵

6.3.3 空间点坐标

```
template<class T>
struct PointXYZ {
    T x, y, z;
};
```

参数:

T	坐标数据类型
x	x 坐标
y	y 坐标
z	z 坐标

6.3.4 点云坐标及相应的 **rgb** 和 **ir** 数据

```
template<class T>
struct PointXYZRgbIr {
    T x, y, z;
    uint8_t r, g, b, ir;
};
```

参数:

T	坐标数据类型
x	x 坐标
y	y 坐标
z	z 坐标
r	red 颜色分量
g	green 颜色分量
b	blue 颜色分量
ir	ir 数据

修订记录

版本	描述	日期	修订者
V1	Initial	15/03/2022	Zhang
V2	Initial	5/9/2023	Yin
V3	增加缺少接口说明	5/29/2023	Yins
V4	心跳功能, GetFrames 更新	19/04/2024	Qin
V5	增加 EnableScanCodeMode 和 SetStreamImageResolution 接口	27/08/2024	Wu
V6	增加设置/获取激光器电流接口	12/18/2024	Wu
V7	优化函数说明, 修改文档格式	03/04/2025	Song

免责声明

本出版物中所述的器件应用信息及其他类似内容仅为您提供便利，它们可能由更新之信息所替代。确保应用符合技术规范，是您自身应负的责任。深圳市光鉴科技有限公司对这些信息不作任何明示或暗示、书面或口头、法定或其他形式的声明或担保，包括但不限于针对其使用情况、质量、性能、适销性或特定用途的适用性的声明或担保。光鉴科技对因这些信息及使用这些信息而引起的后果不承担任何责任。未经深圳市光鉴科技有限公司书面批准，不得将光鉴科技的产品用作生命维持系统中的关键组件。在光鉴科技知识产权保护下，不得暗中或以其他方式转让任何许可证。

技术支持

您可以通过以下途径获得支持：

- 网站支持：访问 www.deptrum.com 获得相关文档和在线支持
- 邮件支持：请将具体问题发邮件至 support@deptrum.com
- FAE 支持：请联系我们的销售人员获取 FAE 的支持方式